Conserving Memory

Large models might cause your machine to run out of memory (OOM). Here are some options that help alleviate this problem.

Tensor Parallelism (TP)

Tensor parallelism (`tensor_parallel_size` option) can be used to split the model across multiple GPUs.

The following code splits the model across 2 GPUs.

```
from vllm import LLM

llm = LLM(model="ibm-granite/granite-3.1-8b-instruct", tensor_parallel_size=2)
```

Warning

To ensure that vLLM initializes CUDA correctly, you should avoid calling related functions (e.g. torch.accelerator.set_device_index) before initializing vLLM. Otherwise, you may run into an error like `RuntimeError: Cannot re-initialize CUDA in forked subprocess`.

To control which devices are used, please instead set the `CUDA_VISIBLE_DEVICES` environment variable.

Note

With tensor parallelism enabled, each process will read the whole model and s
which makes the disk reading time even longer (proportional to the size of tens

Ask AI

You can convert the model checkpoint to a sharded checkpoint using examples/offline_inference/save_sharded_state.py. The conversion process
but later you can load the sharded checkpoint much faster. The model loadin
constant regardless of the size of tensor parallelism.

stable

Quantization

Quantized models take less memory at the cost of lower precision.

Statically quantized models can be downloaded from HF Hub (some popular ones are available at Red Hat AI) and used directly without extra configuration.

Dynamic quantization is also supported via the `quantization` option -- see here for more details.

Context length and batch size

You can further reduce memory usage by limiting the context length of the model (`max_model_len` option) and the maximum batch size (`max_num_seqs` option).

```
from vllm import LLM

llm = LLM(model="adept/fuyu-8b", max_model_len=2048, max_num_seqs=2)
```

Reduce CUDA Graphs

By default, we optimize model inference using CUDA graphs which take up extra memory in the GPU.

You can adjust `compilation_config` to achieve a better balance between inference speed and memory usage:

Code

```
from vllm import LLM
from vllm.config import CompilationConfig, CompilationMode

llm = LLM(
    model="meta-llama/Llama-3.1-8B-Instruct",
    compilation_config=CompilationConfig(
        mode=CompilationMode.VLLM_COMPILE,
        # By default, it goes up to max_num_seqs
        cudagraph_capture_sizes=[1, 2, 4, 8, 16],
    ),
)
```

Ask AI

stable

You can disable graph capturing completely via the `enforce_eager` flag:

```
from vllm import LLM

llm = LLM(model="meta-llama/Llama-3.1-8B-Instruct", enforce_eager=True)
```

Adjust cache size

If you run out of CPU RAM, try the following options:

- (Multi-modal models only) you can set the size of multi-modal cache by setting `mm_processor_cache_gb` engine argument (default 4 GiB).
- (CPU backend only) you can set the size of KV cache using `VLLM_CPU_KVCACHE_SPACE` environment variable (default 4 GiB).

Multi-modal input limits

You can allow a smaller number of multi-modal items per prompt to reduce the memory footprint of the model:

```
from vllm import LLM

# Accept up to 3 images and 1 video per prompt
llm = LLM(
    model="Qwen/Qwen2.5-VL-3B-Instruct",
    limit_mm_per_prompt={"image": 3, "video": 1},
)
```

You can go a step further and disable unused modalities completely by setting its limit to zero. For example, if your application only accepts image input, there is no need to allocate any memory for videos.

```
from vllm import LLM

# Accept any number of images but no videos
llm = LLM(
    model="Qwen/Qwen2.5-VL-3B-Instruct",
    limit_mm_per_prompt={"video": 0},
)
```

Ask AI

You can even run a multi-modal model for text-only inference:

stable

```
from vllm import LLM

# Don't accept images. Just text.
```

```
llm = LLM(
    model="google/gemma-3-27b-it",
    limit_mm_per_prompt={"image": 0},
)
```

Configurable options

`limit_mm_per_prompt` also accepts configurable options per modality. In the configurable form, you still specify `count`, and you may optionally provide size hints that control how vLLM profiles and reserves memory for your multi-modal inputs. This helps you tune memory for the actual media you expect, instead of the model's absolute maxima.

Configurable options by modality:

- `image`: `{"count": int, "width": int, "height": int}`
- `video`: `{"count": int, "num_frames": int, "width": int, "height": int}`
- `audio`: `{"count": int, "length": int}`

Details could be found in `ImageDummyOptions`, `VideoDummyOptions`, and `AudioDummyOptions`.

Examples:

```
from vllm import LLM

# Up to 5 images per prompt, profile with 512x512.
# Up to 1 video per prompt, profile with 32 frames at 640x640.
llm = LLM(
    model="Qwen/Qwen2.5-VL-3B-Instruct",
    limit_mm_per_prompt={
        "image": {"count": 5, "width": 512, "height": 512},
        "video": {"count": 1, "num_frames": 32, "width": 640, "height": 640},
    },
)
```

For backward compatibility, passing an integer works as before and is interpreted as `{"count": <int>}`. For example:

- `limit_mm_per_prompt={"image": 5}` is equivalent to `limit_mm_per_p` `{"count": 5}}`
- You can mix formats: `limit_mm_per_prompt={"image": 5, "video": {"count": 1, "num_frames": 32, "width": 640, "height": 640}}`

Ask AI

stable

Note

- The size hints affect memory profiling only. They shape the dummy inputs used to compute reserved activation sizes. They do not change how inputs are actually processed at inference time.
- If a hint exceeds what the model can accept, vLLM clamps it to the model's effective maximum and may log a warning.

Warning

These size hints currently only affect activation memory profiling. Encoder cache size is determined by the actual inputs at runtime and is not limited by these hints.

Multi-modal processor arguments

For certain models, you can adjust the multi-modal processor arguments to reduce the size of the processed multi-modal inputs, which in turn saves memory.

Here are some examples:

```
from vllm import LLM

# Available for Qwen2-VL series models
llm = LLM(
    model="Qwen/Qwen2.5-VL-3B-Instruct",
    mm_processor_kwargs={"max_pixels": 768 * 768},  # Default is 1280 * 28 * 28
)

# Available for InternVL series models
llm = LLM(
    model="OpenGVLab/InternVL2-2B",
    mm_processor_kwargs={"max_dynamic_patch": 4},  # Default is 12
)
```

March 12, 2026

Ask AI

stable